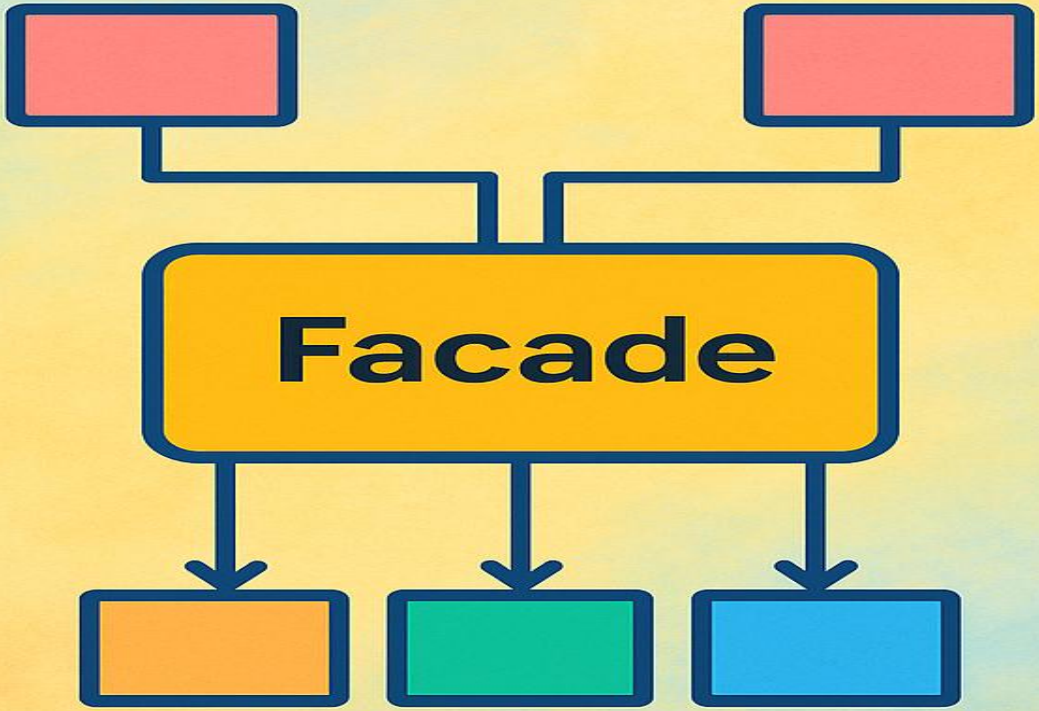
Design Pattern



Creational (5)

-  Factory
- Abstract Factory
-  Singleton
- Prototype
- Builder

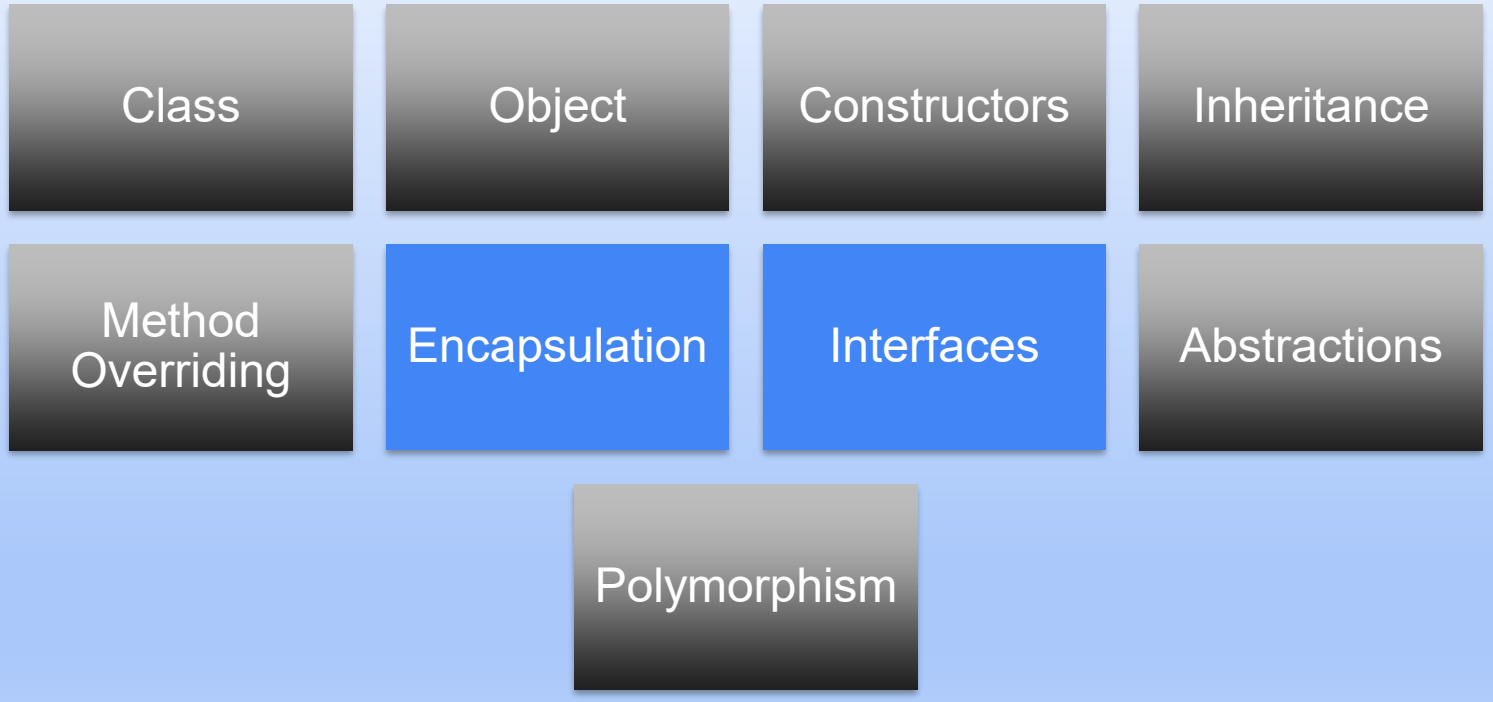
Structural (7)

- Adapter
- Bridge
- Composite
- Decorator
-  Facade
- Flyweight
- Proxy

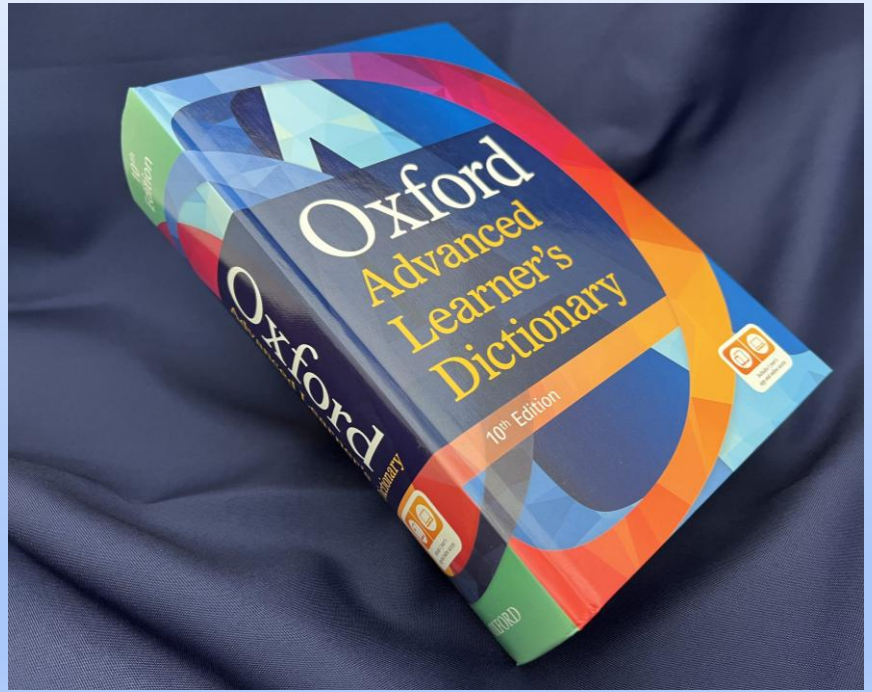
Behavioral (11)

- Chain Of Responsibility
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template
- Visitor

OOP Principals



What is FAÇADE?



- the front of a building.
- the way that somebody/something appears to be, which is different from the way somebody/something really is.

How do you organize a wedding









FACADE Design Pattern



Provide a unified interface to a set of interfaces in a subsystem.



Facade defines a higher-level interface that makes the subsystem easier to use.



Structural type design pattern



Wrapper class that encapsulates a subsystem in order to hide subsystem's complexity.



Can be used in complex systems that have multiple and different sub systems..



Reduce dependencies of outside code on the inner workings of a library.

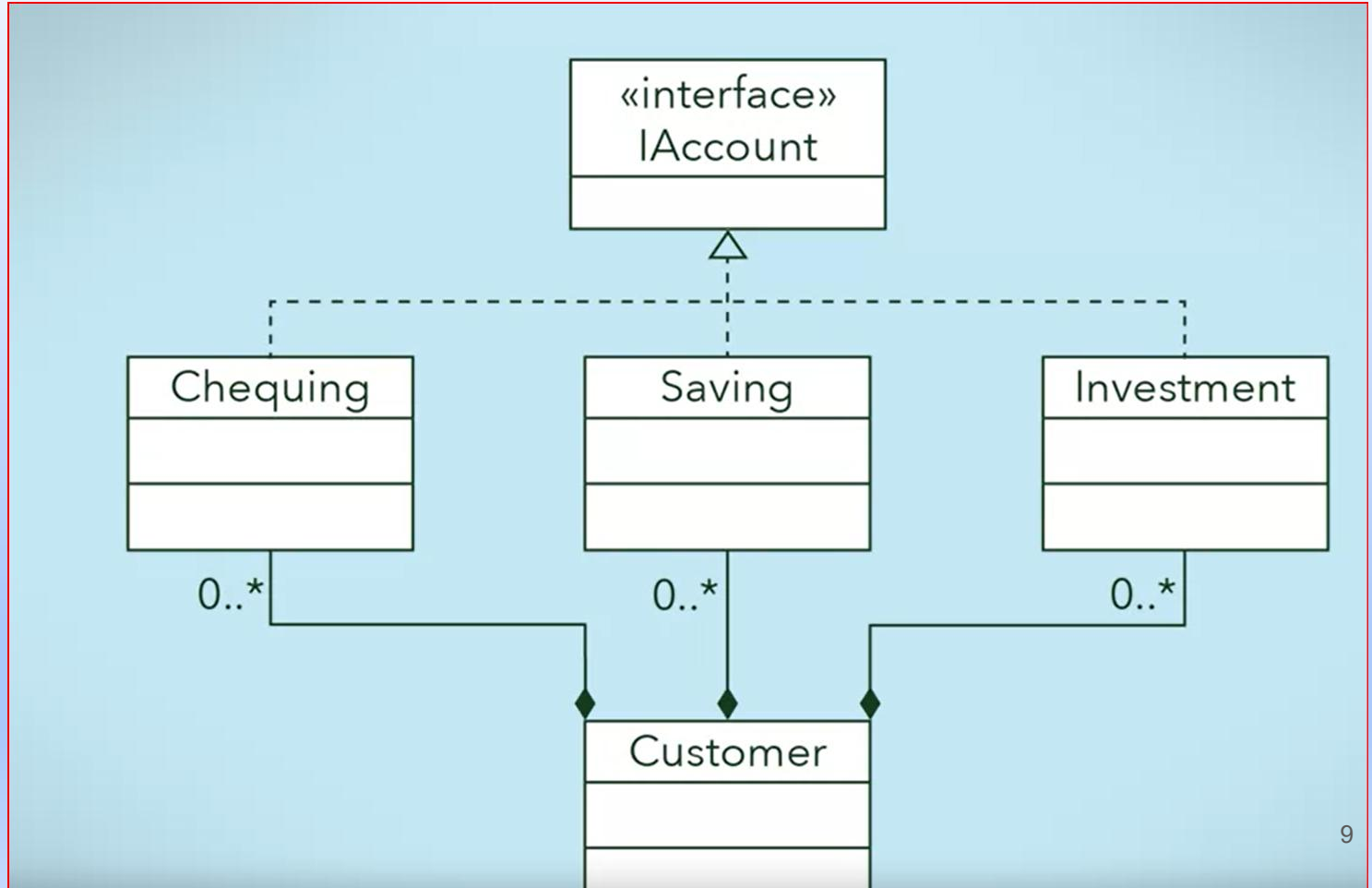
Think about a waiter



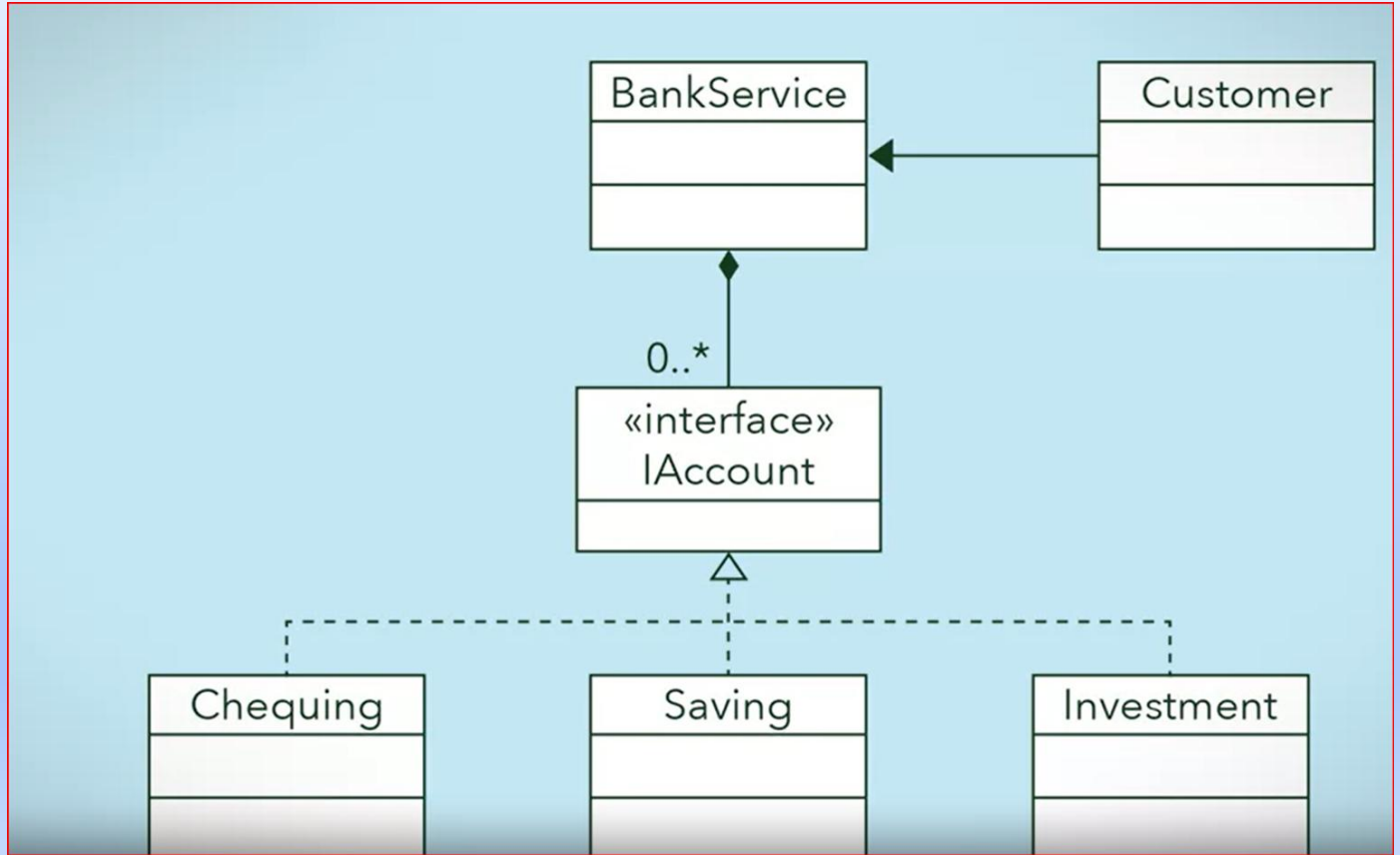


- Facade design pattern does NOT add any functionality. We should try to avoid add any functionally to the interface.
- Facade design pattern is point of entry into system / sub system.
- Facade is a wrapper class that encapsulates a subsystem in order to hide the sub system complexity.
- It interfaces multiple classes to interact.
- User should have the option of directly connect to the subsystems if needed without the interface.

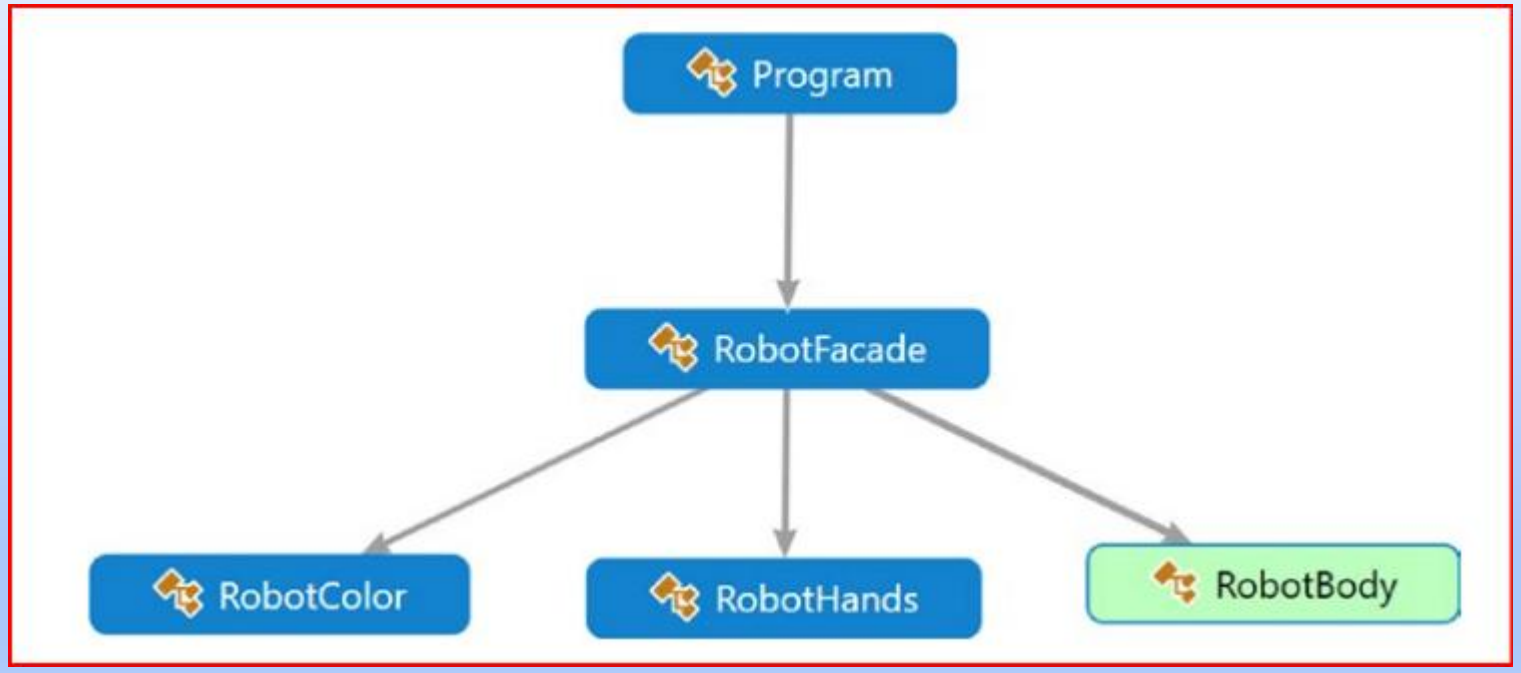
Implementation without FACADE

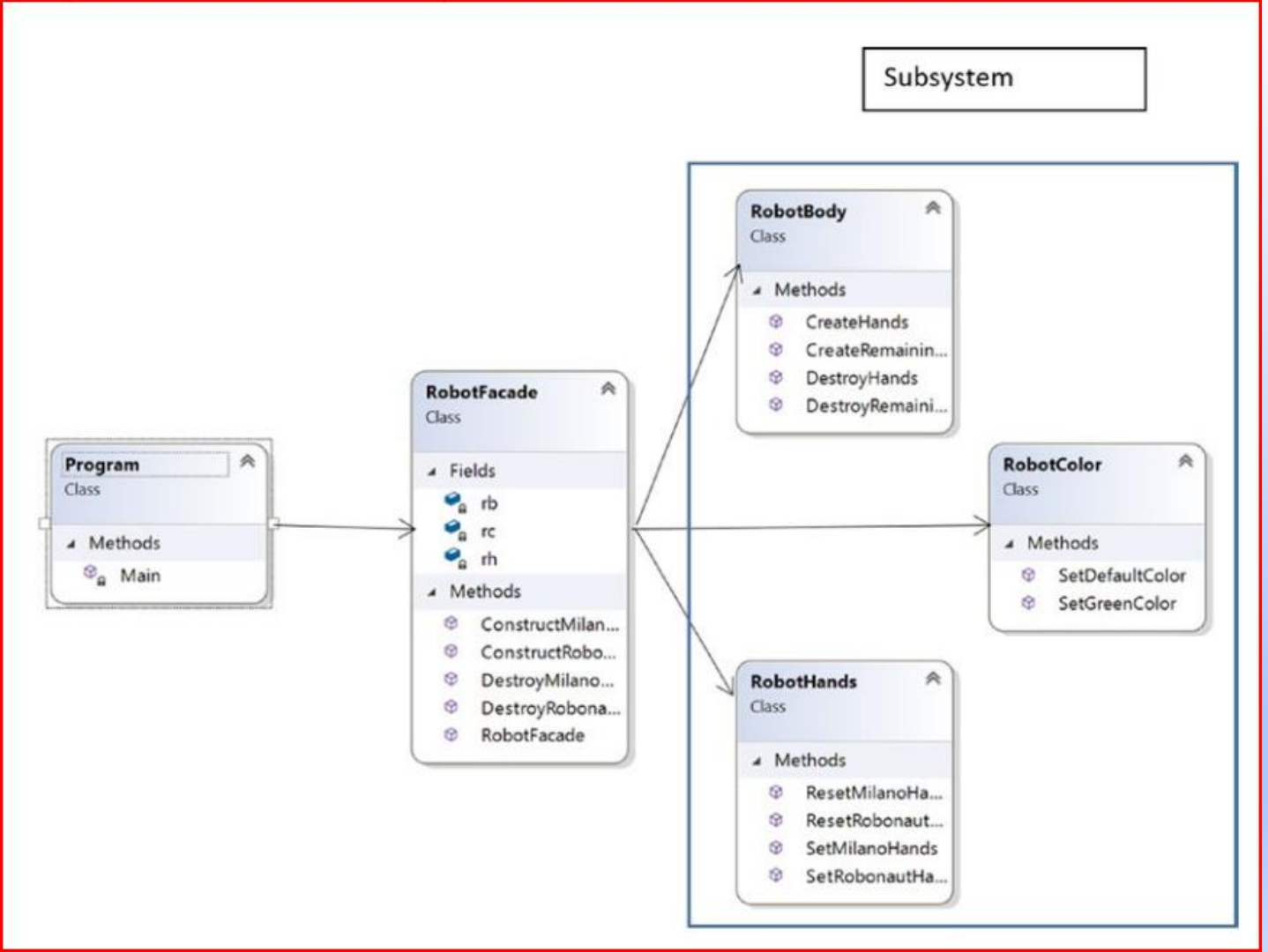


Implementation with FACADE



Refer 116 - 120




Subsystem
Program
Class
Methods
Main
RobotFacade
Class
Fields
rb
rc
rh
Methods
ConstructMilan...
ConstructRobo...
DestroyMilano...
DestroyRobona...
RobotFacade
RobotBody
Class
Methods
CreateHands
CreateRemainin...
DestroyHands
DestroyRemaini...
RobotColor
Class
Methods
SetDefaultColor
SetGreenColor
RobotHands
Class
Methods
ResetMilanoHa...
ResetRobonaut...
SetMilanoHands
SetRobonautHa...

Step 1: Design the interface

```
public interface IAccount {
    public void deposit(BigDecimal amount);
    public void withdraw(BigDecimal amount);
    public void transfer(BigDecimal amount);
    public int getAccountNumber();
}
```

What is an interface class?

- A Java interface is a bit like a class, except a Java interface can only contain method signatures and fields.
- Interfaces will define the specifications for Classes that govern how code will behave.
- An Java interface cannot contain an implementation of the methods, only the signature (name, parameters and exceptions) of the method.
- In interface, variables can be defined only as constants.

```
interface A{
        void m();
        int x=0;
}
```

Step 2: Implement the interface with one or more classes

```
public class Chequing implements IAccount { … }
public class Saving implements IAccount { … }
public class Investment implements IAccount { … }
```

Step 3: Create the facade class and wrap the classes that implements the interface

```
public class BankService {
	private Hashtable<int, IAccount> bankAccounts;
	public BankService() {
		this.bankAccounts = new Hashtable<int, IAccount>;
	}
	public int createNewAccount(String type, BigDecimal initAmount) {
		IAccount newAccount = null;
		switch (type) {
			case "chequing":
				newAccount = new Chequing(initAmount);
				break;
			case "saving":
				newAccount = new Saving(initAmount);
				break;
			case "investment":
				newAccount = new Investment(initAmount);
				break;
			default:
				System.out.println("Invalid account type");
				break;
		}
		if (newAccount != null) {
			this.bankAccounts.put(newAccount.getAccountNumber(), newAccount);
			return newAccount.getAccountNumber();
		}
		return -1;
	}
	public void transferMoney(int to, int from, BigDecimal amount) {
		IAccount toAccount = this.bankAccounts.get(to);
		IAccount fromAccount = this.bankAccounts.get(from);
		fromAccount.transfer(toAccount, amount);
```

```
public class Customer {

    public static void main(String args[]) {
        BankService myBankService = new BankService();

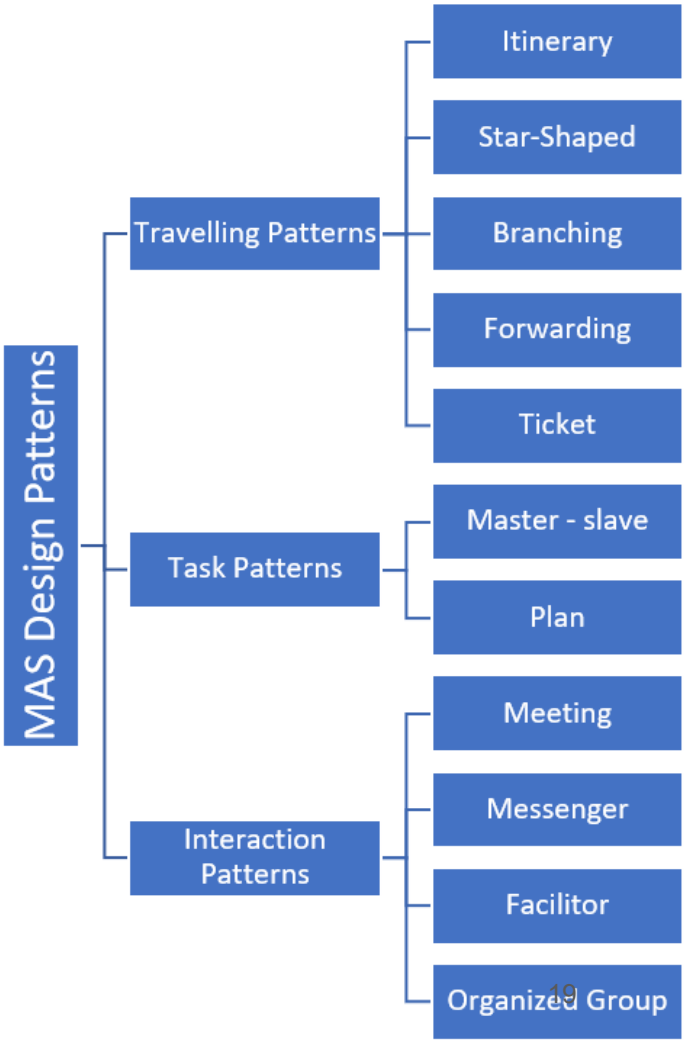
        int mySaving = myBankService.createNewAccount("saving",
            new BigDecimal(500.00));

        int myInvestment = myBankService.createNewAccount(
            "investment", new BigDecimal(1000.00));

        myBankService.transferMoney(mySaving, myInvestment, new
            BigDecimal(300.00));
    }

}
```

Extension of Facade Design Patterns-Multi Agents



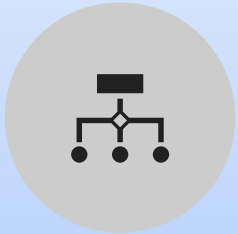
Facade Design Pattern

- Use to hide the complexity of a subsystem by encapsulating it behind a unified wrapper class called a facade class.
- Removes the need for client classes to manage a subsystem on their own
- Handles instantiation and redirection of tasks to appropriate class within the subsystem.
- Provides client classes with a simplified interface for the subsystem.
- Acts simply as a point of entry to a substem and does not add more functional to the subsystem.

Q&A



WHAT ARE THE KEY ADVANTAGES OF USING THE FACADE PATTERN?



CAN YOU ACCESS EACH OF THE SUBSYSTEMS DIRECTLY?



THERE SHOULD BE ONLY ONE FACADE FOR A COMPLEX SUBSYSTEM. IS THIS UNDERSTANDING CORRECT?



WHAT ARE THE CHALLENGES ASSOCIATED WITH THE FACADE PATTERN?

How FAÇADE Design Pattern Can Be Used in Your Projects?